

Aplicación del Traveling Salesman Problem (TSP) para planeamiento de rutas de distribución

Tomás Ruíz, Jaime Gamba, Juan Felipe Poveda, Juan José Obando

I. INTRODUCCIÓN

La teoría de grafos es una de las ramas más importantes de las matemáticas discretas, cuyos resultados son relevantes en las ciencias de la computación, naturales y sociales [1]. Es una herramienta muy útil para representar relaciones entre entidades de sistemas biológicos, físicos, matemáticos, sociales y logísticos. Su origen aparece en el planteamiento del problema de *los puentes de Königsberg* planteado por Leonhard Euler [1].

El problema del viajante, referido como TSP por las siglas de *"traveling salesman problem"*, es un problema que se puede abordar por medio de la teoría de grafos y ha sido investigado extensamente; se han escrito una multitud de artículos de investigación y libros con respecto a este tema. Las soluciones a este problema tienen muchas aplicaciones en el mundo real. Por ejemplo:

- Cableado de computadoras
- Recorte de papel tapiz
- Perforación en contextos industriales
- Secuenciación de procesos
- Cristalografía

son algunas de las aplicaciones del problema [2]. El TSP es caracterizado por tener un planteamiento que puede parecer sencillo: *dado un conjunto de ciudades, encontrar un recorrido que visite todas exactamente una vez y al final regrese al punto de origen*. Sin embargo, solucionarlo no resulta igual de fácil, pues es un problema *NP-hard*. Esto significa que no se conoce un algoritmo que lo solucione en complejidad temporal polinomial, por lo que las soluciones al problema en la vida real se ven obligadas a elegir entre heurísticas y aproximaciones antes que algoritmos exactos, dado que se manipula una gran cantidad de información y los algoritmos que encuentran soluciones exactas crecen exponencialmente [2].

Este proyecto busca hacer una investigación y comparación de los diversos algoritmos que usan la teoría de grafos para resolver el TSP con la finalidad de aplicarlo a un problema real: la planeación de rutas para sistemas de distribución bajo criterios simultáneos. Se modelará el problema creando un grafo que sea congruente con la situación y las necesidades del problema utilizando datos reales extraídos de fuentes de información geográfica, y se realizará un análisis estructural del grafo generado. Finalmente, se realizará una herramienta computacional que solucione el problema basado en todo el análisis previo.

II. DESCRIPCIÓN DEL PROBLEMA

Nuestro grupo está trabajando en un proyecto llamado *FarmaGo*, el cual consiste en poner máquinas expendedoras de medicamentos en espacios públicos, por ejemplo, aeropuertos, centros comerciales o parques, donde sea beneficioso tener acceso fácil y urgente a fármacos de venta libre o a artículos de higiene. Naturalmente, se necesita tener un sistema para abastecer estas máquinas expendedoras, de modo que los medicamentos idealmente siempre estén disponibles para quien los necesite. Tal sistema requiere transportar los medicamentos a lo largo de la ciudad, y el transporte tiene un costo que se desea que sea lo más pequeño posible. Hay muchos factores que entran en juego para decidir el costo de las tarifas de transporte: ubicación geográfica, tamaño de la carga y factores regulatorios y normativos. Sin embargo, los que son directamente afectados por la escogencia de la ruta son la distancia, tiempo de tránsito y congestión vehicular. En lo posible, se deben minimizar todos estos factores a la vez y, en el caso de que no se pueda, priorizar algunos factores sobre otros basado en los *trade-offs* que hay entre ellos. Una restricción que se puede plantear para la ruta (por más evidente que parezca) es que la ruta pase por cada uno de los puntos donde se debe entregar el medicamento una sola vez. Si esto no se asegura, la ruta planificada estaría haciendo recorridos innecesarios al ir a ubicaciones donde ya se repartió el medicamento.

III. OBJETIVOS GENERALES Y ESPECÍFICOS

OBJETIVOS GENERALES

- Diseñar un sistema de planeación de rutas que encuentre rutas óptimas para la distribución de medicamentos, minimizando múltiples criterios de manera simultánea y que use datos del mundo real.

OBJETIVOS ESPECÍFICOS

- Modelar el problema mediante un grafo ponderado, donde las ubicaciones de distribución son vértices y las aristas contienen varios datos relacionados a las carreteras entre vértices, como distancia, tiempo de transporte y congestión vial.
- Obtener los valores del grafo de datos reales extraídos de fuentes públicas como APIs de datos geoespaciales.
- Realizar un análisis del grafo que modela la situación por medio del cálculo de métricas como grados de los vértices, conectividad y densidad del grafo para determinar la complejidad del mismo.

- Comparar varios algoritmos y estrategias para encontrar una solución suficientemente buena al TSP.
- Mediante las soluciones obtenidas al TSP, representar posibles rutas para luego determinar la viabilidad, ventajas y desventajas de cada uno haciendo uso de medidas de complejidad computacional en las comparaciones.

IV. MARCO TEÓRICO

A. IV-A TSP

- *NP*: Nondeterministic polynomial es una clase de problemas la cual, dada una solución, se puede verificar si es calculable y correcta en tiempo polinomial o rápido; además, estos problemas se pueden clasificar según su complejidad en subclases como NP-completo y NP-hard, las cuales agrupan los problemas más difíciles dentro y relacionados con NP [3].
- *NP-complete*: Problemas dentro de NP que son los más difíciles; si al menos uno se resuelve rápidamente, todos los NP restantes también [3].
- *NP-hard*: Problemas que son al menos tan difíciles como los complete, pero no se pueden verificar rápidamente [3].
- *Inviabilidad computacional*: Un problema es inviable computacionalmente si no hay un algoritmo con tiempo polinomial conocido que lo pueda resolver. Estos problemas generalmente requieren un tiempo de $O(2^n)$ o $O(n!)$, volviéndolos así imprácticos [4].
- *TSP*: El traveling salesman problem (TSP) o, en su traducción, problema del viajante, es el artículo clásico de optimización que consiste en encontrar cuál de todas las rutas posibles en un país es la que visita cada ciudad exactamente una vez y regresa a la ciudad origen. Este problema puede modelarse en un grafo ponderado donde los vértices son las ciudades y las aristas son el costo incluido en el desplazamiento, representando tiempo, distancia o congestión entre ellas. El TSP es un problema NP-hard [5].
- *Ciclo Hamiltoniano*: Un ciclo Hamiltoniano es un camino cerrado que visita todos los vértices del grafo una única vez y que finaliza en el mismo vértice en el que inicia. Además, un grafo es Hamiltoniano si contiene un ciclo Hamiltoniano.
- *TSP simétrico vs asimétrico*: Dentro de un TSP asimétrico, el costo del punto A al B puede ser distinto que de B a A. Es decir:

$$w(i, j) \neq w(j, i) \quad (1)$$

Un problema de TSP simétrico implica que el costo del punto A al B es exactamente el mismo que del punto B al A. Es decir:

$$w(i, j) = w(j, i) \quad (2)$$

[1]

B. IV-A Algoritmos

- *Algoritmos Voraces*: Los algoritmos voraces (greedy) son una estrategia de diseño de algoritmos que busca

soluciones óptimas teniendo en cuenta la mejor decisión posible en el paso actual, sin tener en cuenta las decisiones previas. Estos cumplen con la particularidad de que son eficientes y rápidos aunque no garanticen la solución óptima absoluta [6].

- *Algoritmos Heurísticos*: Los algoritmos heurísticos son una estrategia de diseño de algoritmos que busca soluciones suficientemente buenas en un tiempo razonable. Esta estrategia intenta mantener un balance entre precisión-optimalidad y velocidad. Esta estrategia es principalmente usada para solucionar problemas NP-hard donde la solución óptima es inviable computacionalmente [7].
- *Algoritmos Geométricos*: Los algoritmos geométricos son una estrategia de diseño que tiene en cuenta una secuencia lógica de pasos para resolver problemas relacionados con espacios, formas y trayectorias [8].
- *Algoritmos Exactos*: Los algoritmos exactos son otra estrategia de diseño de algoritmos que buscan encontrar la solución óptima absoluta para un determinado problema. Esto siempre garantiza el mejor resultado posible sin tener en cuenta si es computacionalmente viable. Esta estrategia se asegura de recorrer todas las posibles soluciones para luego compararlas y llegar al óptimo absoluto [9].
- *Nearest Neighbor algorithm*: El algoritmo del vecino más cercano es un tipo de algoritmo voraz, lo que significa que en cada etapa del algoritmo se elige la mejor arista posible. En otras palabras, se toma una decisión localmente óptima en cada paso del algoritmo. Otro aspecto a tener en cuenta sobre este algoritmo es que se necesita elegir un vértice inicial, y si se eligen vértices diferentes, se pueden obtener ciclos hamiltonianos distintos [5].

Algorithm 1 Algoritmo del Vecino Más Cercano

Input: Grafo G : Grafo de entrada.

Output: Ciclo Hamiltoniano aproximado.

Iteraciones: 1. Inicialización:

- Escoger un vértice $u_i \in V(G)$.
- Formar un camino $P = \{u_i\}$.
- Definir $T = V(P)$ (conjunto de nodos visitados).
- Definir $r = 1$ y $v_r = u_i$.

2. Iteración:

- Repetir hasta que $T = V(G)$:
 - * Escoger $u_j \in V(G) \setminus T$ tal que la distancia desde v_r sea mínima.
 - * (En caso de empate, seleccionar arbitrariamente).
 - * Definir $v_{r+1} = u_j$ y actualizar $P = P \cup \{u_j\}$.
 - * Actualizar $T = T \cup \{u_j\}$.
 - * Incrementar $r = r + 1$.

3. Resultado:

- Formar un ciclo Hamiltoniano conectando el último vértice de P con el primero.

[5]

- *Closest Insertion Algorithm*: Es una heurística que va construyendo un ciclo de manera incremental. Inicia for-

mando un ciclo con un subconjunto pequeño de vértices, para que en cada iteración se seleccione el vértice no incluido que se encuentra más cercano a alguno de los vértices unidos; este proceso se repite hasta tener conectados todos los vértices con sus costos correspondientes [5].

Algorithm 2 Algoritmo de Inserción Más Cercana

Input: Grafo G : Grafo completo ponderado.

Output: Ciclo Hamiltoniano aproximado.

Iteraciones:

1. Inicialización:

- Seleccionar un subconjunto inicial de vértices y formar un ciclo C .

2. Iteración:

- Repetir hasta que todos los vértices estén en C :
 - * Seleccionar el vértice $u \notin C$ más cercano a algún vértice en C .
 - * Encontrar la arista (v_i, v_j) en C donde insertar u minimice el incremento del costo:

$$\Delta = w(v_i, u) + w(u, v_j) - w(v_i, v_j)$$

- * Insertar u entre v_i y v_j en el ciclo C .

3. Resultado:

- Retornar el ciclo C como solución aproximada.

[5]

- *Geometric Algorithm*: Es un método heurístico que sirve cuando los vértices representan puntos en el plano. Este algoritmo utiliza propiedades geométricas para construir una ruta eficiente, comenzando generalmente con el cierre convexo del conjunto de puntos y luego insertando los vértices restantes en posiciones que minimicen el incremento del costo; por lo cual también busca reducir intersecciones entre aristas, haciéndolo más eficiente [5].

Algorithm	3	Algoritmo	Geométrico
------------------	----------	------------------	-------------------

Input: Conjunto de puntos en el plano (coordenadas).

Output: Ciclo Hamiltoniano aproximado. **Iteraciones:**

1. Inicialización:

- Construir el cierre convexo (convex hull) del conjunto de puntos.
- Definir el ciclo inicial C con los puntos del cierre convexo.

2. Iteración:

- Repetir hasta incluir todos los puntos:
 - * Seleccionar un punto p que no esté en el ciclo C .
 - * Determinar la arista (v_i, v_j) en C donde insertar p minimice el incremento del costo.
 - * Insertar p en el ciclo C .

3. Optimización:

- Revisar el ciclo y eliminar intersecciones entre aristas si existen.

4. Resultado:

- Retornar el ciclo C como solución aproximada.

[5]

- *Algoritmo Calcular distancias*: Es un algoritmo para calcular el costo total de una ruta x en el TSP entre nodos considerando el retorno [10].

Algorithm 4 Función Calcular Distancia Total

Input:

- Ruta π : Permutación de los vértices.
- Matriz de distancias D (simétrica).

Output:

- Distancia total del recorrido.

Iteraciones: 1. Inicialización:

- $total = 0$

2. Recorrido de la ruta:

- Para $i = 0$ hasta $|\pi| - 2$:
 - * $total = total + D[\pi[i]][\pi[i + 1]]$

3. Cierre del ciclo:

- $total = total + D[\pi[|\pi| - 1]][\pi[0]]$

4. Resultado:

- Retornar $total$

[10]

- *Algoritmo Fuerza Bruta para TSP*: Es un algoritmo para evaluar todas las posibles permutaciones de rutas, garantizando encontrar la solución óptima, pero no es muy bueno para instancias grandes debido a su costo computacional factorial [10].

Algorithm 5 Algoritmo de Fuerza Bruta para TSP

Input:

- Matriz de distancias D de tamaño $n \times n$, tal que $D[i][j] = D[j][i]$.

Output:

- Ruta óptima π .
- Costo mínimo asociado.

Iteraciones:

1. Inicialización:

- $n = |D|$
- $ciudades = \{1, 2, 3, \dots, n - 1\}$
- $mejorRuta = \emptyset$
- $mejorCosto = \infty$

2. Generación de soluciones:

- Para cada permutación p de $ciudades$ tal que $p \neq reverse(p)$:
 - * Construir $rutaActual = \{0\} \cup p$
 - * Calcular $costo = CalcularDistancias(rutaActual, D)$
 - * Si $costo < mejorCosto$:
 - $mejorCosto = costo$
 - $mejorRuta = rutaActual$

3. Cierre del ciclo:

- Agregar el nodo inicial 0 al final de $mejorRuta$

4. Resultado:

- Retornar (*mejorRuta, mejorCosto*)

[10]

- **Algoritmo 2-OPT:** Es un algoritmo de búsqueda local que busca reducir la distancia total de una ruta eliminando cruces mediante iteraciones de reordenamiento; consiste en seleccionar 2 aristas, eliminarlas y reconectar de forma inversa los nodos para "desenredar" el camino. Esto se repite hasta que no se puedan hacer más mejoras alcanzando el óptimo local [11].

Algorithm 6 Algoritmo 2-OPT

Input:

- Grafo completo con distancias definidas en las aristas.
- Una ruta inicial y su distancia.

Output:

- Ruta optimizada mediante 2-opt.

Iteraciones:

1. Repetición:

- Repetir hasta que no haya mejora:

2. Búsqueda de mejoras:

- Para cada i en nodos elegibles:
 - * Para cada j en nodos elegibles tal que $j > i$:
 - Aplicar intercambio 2-opt entre i y j :
 - Tomar la ruta desde el inicio hasta i en orden.
 - Tomar la subruta entre i y j e invertirla.
 - Tomar la ruta desde j hasta el final en orden.
 - Calcular la nueva distancia.
 - Si la nueva distancia es menor:
 - Actualizar la ruta con el nuevo orden.

3. Resultado:

- Retornar la ruta optimizada.

[11]

V. MODELAMIENTO DEL PROBLEMA

Para modelar el problema, se utilizará un grafo no dirigido ponderado donde los nodos representan puntos donde hay una máquina que se debe abastecer. Una arista entre dos vértices u y v representa que quien distribuye la mercancía a los puntos puede llegar físicamente del punto u al v o del v al u de alguna manera (para este problema no es relevante cómo). Dentro del planteamiento del problema, se asume que no hay problema en que el distribuidor se transporte físicamente de un punto a otro, podemos definir el grafo como un grafo completo. Adicionalmente, como el grafo es ponderado, se modela el peso de una arista uv (denotado $w(u, v)$) como el costo de que el distribuidor se transporte de u a v . El problema requiere varias métricas a optimizar, puntualmente:

- Distancia entre los puntos de distribución.
- Congestión vial entre los puntos de distribución.
- Tiempo estimado de transporte entre los puntos de distribución.

Por lo que el costo debe tomar las 3 métricas en consideración. Debido a esto, se modela el costo de u a v de la siguiente manera:

$$w(u, v) = \alpha \cdot a(u, v) + \beta \cdot b(u, v) + \gamma \cdot c(u, v) \quad (3)$$

Donde:

- $a(u, v) :=$ Distancia entre los puntos de distribución u y v en km.
- $b(u, v) :=$ Congestión vehicular entre los puntos u y v (cantidad adimensional entre 0 y 1).
- $c(u, v) :=$ Tiempo estimado de transporte entre los puntos de distribución en minutos.
- $\alpha :=$ peso de la métrica $a(u, v)$.
- $\beta :=$ peso de la métrica $b(u, v)$.
- $\gamma :=$ peso de la métrica $c(u, v)$.

Se optó por esta forma de calcular el costo para tener en consideración múltiples métricas en el proceso de optimización, pero también para parametrizar la importancia de cada una, de modo que si se requiere darle más importancia a unas métricas sobre otras, se pueda hacer esto modificando los pesos asignados a cada una de las métricas. Note que en la práctica, las 3 métricas tienen que ser normalizadas, de modo que todas aporten al costo en las mismas unidades y la escogencia de pesos en el costo tenga sentido.

VI. SOLUCIÓN PROPUESTA

La solución propuesta a este problema se implementará en Python. Se utilizará la librería NetworkX que tiene varias utilidades para crear y manipular grafos, junto a varios algoritmos que se pueden ejecutar sobre ellos. Para construir el grafo, se tendrá una lista de ubicaciones y su correspondiente información, con la cual se puede llamar a la API de Google Maps para obtener los datos de distancia y tiempo. Luego, se llamará a la API de TomTom Traffic para obtener la información sobre la cantidad de tráfico que hay de cada punto de distribución a otro. Ya con todo esto recopilado, se construirá el grafo en NetworkX. Al tener el grafo construido, se realizará un análisis sobre él. Este consiste en observar la distribución de pesos mediante un histograma para ver en qué valor se concentran los pesos dentro del grafo. También se pueden calcular medidas como la varianza para identificar qué tanto se alejan los pesos del valor típico. Luego, se ejecutarán los algoritmos descritos anteriormente (si NetworkX no tiene una implementación de los algoritmos, antes de ejecutarse también se implementarán). Se ejecutarán para distintos tamaños de grafos para ver cómo escalan, se medirá su tiempo de ejecución y se comparará la calidad de las soluciones que estos ofrecen.

REFERENCES

- [1] D. B. West, *Introduction to Graph Theory*, 2nd ed. Pearson, 1996.
- [2] G. Laporte, "The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization," *Journal of the Operational Research Society*, vol. 37, no. 5, pp. 535–536, 5 1986. [Online]. Available: <https://doi.org/10.1057/jors.1986.93>
- [3] M. Pokharel, "Computational complexity theory(p.np,np-complete and np-hard problems)," 06 2020.

- [4] f. given i=HK, given=Hemanta K. On computational intractability assumptions in cryptography. [Online]. Available: <https://www.ideals.illinois.edu/items/29958#:~:text=the%20existence%20of%20some%20puzzle,intractability%20assumptions>
- [5] C. Brucato, "THE TRAVELING SALESMAN PROBLEM," Ph.D. dissertation, University of Pittsburgh, 2013. [Online]. Available: <https://www.mathematics.pitt.edu/sites/default/files/TSP.pdf>
- [6] g. given i=G. Greedy algorithms tutorial. [Online]. Available: <https://www.geeksforgeeks.org/dsa/greedy-algorithms/%7D>
- [7] Heuristic algorithms - cornell university computational optimization open textbook - optimization wiki. [Online]. Available: https://optimization.cbe.cornell.edu/index.php?title=Heuristic_algorithms
- [8] g. given i=G. Geometric algorithms. [Online]. Available: <https://www.geeksforgeeks.org/dsa/geometric-algorithms/%7D>
- [9] f. given i=FV, given=Fedor V. and f. given i=P, given=Petteri, "Exact exponential algorithms," vol. 56, no. 3, pp. 80–88. [Online]. Available: <https://doi.org/10.1145/2428556.2428575%7D>
- [10] University of Waterloo. (n.d.) Concorde home: Documentation. Accessed: 2026-04-18. [Online]. Available: <https://www.math.uwaterloo.ca/tsp/concorde/DOC/index.html>
- [11] J. J. A. Slootbeek, "Average-case analysis of the 2-opt heuristic for the tsp," Master Thesis, University of Twente, Enschede, Netherlands, 2026, supervisor: Dr. B. Manthey. Chair of Discrete Mathematics and Mathematical Programming.